

Vitistack

🚀 Kubernetes clusters, on multiple machine and kubernetes providers

🌐 vitistack.io · 📖 docs.vitistack.io · 🐙 github.com/vitistack

What is Vitistack? 🤔



What is Vitistack? 🤔



- 🏗️ An **infrastructure platform** for running Kubernetes with many machine-providers and kubernetes-providers

What is Vitistack? 🤔



- 🏗️ An **infrastructure platform** for running Kubernetes with many machine-providers and kubernetes-providers
- 📜 Kubernetes clusters as **declarative resources** (CRDs)

What is Vitistack? 🤔



- 🏗️ An **infrastructure platform** for running Kubernetes with many machine-providers and kubernetes-providers
- 📜 Kubernetes clusters as **declarative resources** (CRDs)
- 🤖 Built from **operators** – each does one job well

What is Vitistack? 🤔



- 🏗️ An **infrastructure platform** for running Kubernetes with many machine-providers and kubernetes-providers
- 📜 Kubernetes clusters as **declarative resources** (CRDs)
- 🤖 Built from **operators** – each does one job well
- 🔓 Open source building blocks

What is Vitistack? 🤔



- 🏗️ An **infrastructure platform** for running Kubernetes with many machine-providers and kubernetes-providers
- 📜 Kubernetes clusters as **declarative resources** (CRDs)
- 🤖 Built from **operators** – each does one job well
- 🔓 Open source building blocks
- 🎯 Goals: cost-efficiency, flexibility, security, interoperability, customizable

What is Vitistack? 🤔



- 🏗️ An **infrastructure platform** for running Kubernetes with many machine-providers and kubernetes-providers
- 📜 Kubernetes clusters as **declarative resources** (CRDs)
- 🤖 Built from **operators** – each does one job well
- 🔓 Open source building blocks
- 🎯 Goals: cost-efficiency, flexibility, security, interoperability, customizable

🧱 Building blocks

Area	Examples
Kubernetes	**Talos**, **AKS**, K0s, Scaleway,etc
🖥️ Machines	**KubeVirt**, **Proxmox**, (future Physical)
🌐 Network / IP	**Static IP**, **Kea**, **Custom**
💾 Storage	**Ceph**, Longhorn, etc

Kubernetes operators – control loop pattern 🔁



🎯 DESIRED STATE · SPEC

```
nodePools[wp].replicas: 5
```

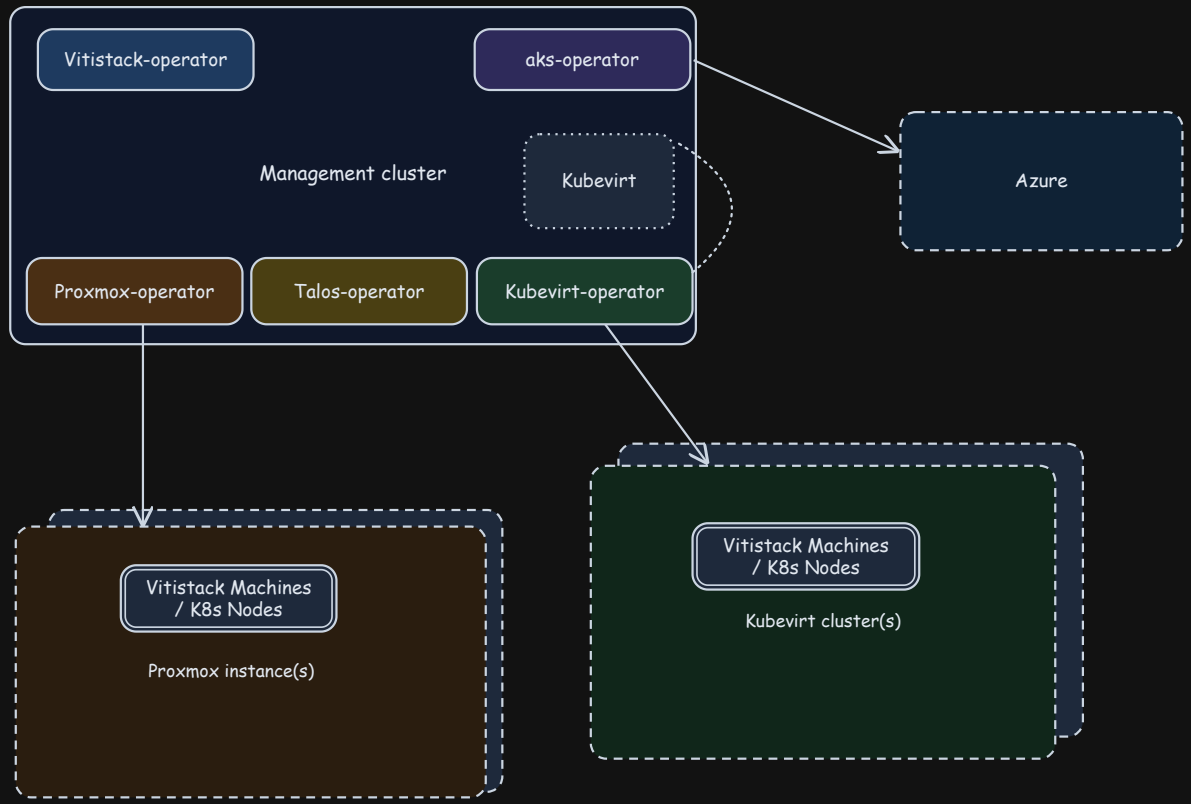
🌍 ACTUAL STATE · THE REAL WORLD



✅ **In sync: status updated**

🤖 Operator = **CRD** (the API) + **controller** (the loop)

Clusters and Operators





How to create a cluster 🛠️

 `NetworkNamespace` →  `KubernetesCluster`

NetworkNamespace

Where the cluster(s) lives on the network

```yaml
apiVersion: vitistack.io/v1alpha2
kind: NetworkNamespace
metadata:
  name: dev-net-001
  namespace: team-cool
spec:
  ipAllocation:
    provider: static-ip-operator
    type: static
  networkProvisioning:
    provider: nam # custom network provider
```

NetworkNamespace

Where the cluster(s) lives on the network

```yaml
apiVersion: vitistack.io/v1alpha2
kind: NetworkNamespace
metadata:
  name: dev-net-001
  namespace: team-cool
spec:
  ipAllocation:
    provider: static-ip-operator
    type: static
  networkProvisioning:
    provider: nam # custom network provider
```

apiVersion / kind

the Vitistack API and the resource type

NetworkNamespace

Where the cluster(s) lives on the network

```yaml
apiVersion: vitistack.io/v1alpha2
kind: NetworkNamespace
metadata:
  name: dev-net-001
  namespace: team-cool
spec:
  ipAllocation:
    provider: static-ip-operator
    type: static
  networkProvisioning:
    provider: nam # custom network provider
```

apiVersion / kind

the Vitistack API and the resource type

name / namespace

one per team, clusters reference it by name

NetworkNamespace

Where the cluster(s) lives on the network

```yaml
apiVersion: vitistack.io/v1alpha2
kind: NetworkNamespace
metadata:
  name: dev-net-001
  namespace: team-cool
spec:
  ipAllocation:
    provider: static-ip-operator
    type: static
  networkProvisioning:
    provider: nam # custom network provider
```

apiVersion / kind
the Vitistack API and the resource type

name / namespace
one per team, clusters reference it by name

ipAllocation
who hands out IPs (static-ip-operator, static)

NetworkNamespace

Where the cluster(s) lives on the network

```yaml
apiVersion: vitistack.io/v1alpha2
kind: NetworkNamespace
metadata:
  name: dev-net-001
  namespace: team-cool
spec:
  ipAllocation:
    provider: static-ip-operator
    type: static
  networkProvisioning:
    provider: nam # custom network provider
```

apiVersion / kind
the Vitistack API and the resource type

name / namespace
one per team, clusters reference it by name

ipAllocation
who hands out IPs (static-ip-operator, static)

networkProvisioning
who provisions the network (nam)

KubernetesCluster – `spec.data`

 Identity and 📍 placement

```yaml
apiVersion: vitistack.io/v1alpha1
kind: KubernetesCluster
metadata:
  name: supercluster
spec:
  data:
    clusterId: supercluster-1337
    clusterUid: ce660a80-054e-419f-b22c-b6c66a9710bd
    environment: dev
    networkNamespaceName: dev-net-001
    provider: talos
    datacenter: no-south-az1
    region: south
    zone: az1
    project: simple-project
    workorder: simple-workorder
    workspace: simple-workspace
```

KubernetesCluster – `spec.data`

 Identity and 📍 placement

```yaml
apiVersion: vitistack.io/v1alpha1
kind: KubernetesCluster
metadata:
  name: supercluster
spec:
  data:
    clusterId: supercluster-1337
    clusterUid: ce660a80-054e-419f-b22c-b6c66a9710bd
    environment: dev
    networkNamespaceName: dev-net-001
    provider: talos
    datacenter: no-south-az1
    region: south
    zone: az1
    project: simple-project
    workorder: simple-workorder
    workspace: simple-workspace
```

KubernetesCluster – `spec.data`

 Identity and 📍 placement

```yaml
apiVersion: vitistack.io/v1alpha1
kind: KubernetesCluster
metadata:
  name: supercluster
spec:
  data:
    clusterId: supercluster-1337
    clusterUid: ce660a80-054e-419f-b22c-b6c66a9710bd
    environment: dev
    networkNamespaceName: dev-net-001
    provider: talos
    datacenter: no-south-az1
    region: south
    zone: az1
    project: simple-project
    workorder: simple-workorder
    workspace: simple-workspace
```

KubernetesCluster – `spec.data`

Identity and placement

```yaml
apiVersion: vitistack.io/v1alpha1
kind: KubernetesCluster
metadata:
  name: supercluster
spec:
  data:
    clusterId: supercluster-1337
    clusterUid: ce660a80-054e-419f-b22c-b6c66a9710bd
    environment: dev
    networkNamespaceName: dev-net-001
    provider: talos
    datacenter: no-south-az1
    region: south
    zone: az1
    project: simple-project
    workorder: simple-workorder
    workspace: simple-workspace
```

KubernetesCluster – `spec.data`

Identity and placement

```yaml
apiVersion: vitistack.io/v1alpha1
kind: KubernetesCluster
metadata:
  name: supercluster
spec:
  data:
    clusterId: supercluster-1337
    clusterUid: ce660a80-054e-419f-b22c-b6c66a9710bd
    environment: dev
    networkNamespaceName: dev-net-001
    provider: talos
    datacenter: no-south-az1
    region: south
    zone: az1
    project: simple-project
    workorder: simple-workorder
    workspace: simple-workspace
```

KubernetesCluster – `spec.data`

🪪 Identity and 📍 placement

```yaml
apiVersion: vitistack.io/v1alpha1
kind: KubernetesCluster
metadata:
  name: supercluster
spec:
  data:
    clusterId: supercluster-1337
    clusterUid: ce660a80-054e-419f-b22c-b6c66a9710bd
    environment: dev
    networkNamespaceName: dev-net-001
    provider: talos
    datacenter: no-south-az1
    region: south
    zone: az1
    project: simple-project
    workorder: simple-workorder
    workspace: simple-workspace
```

KubernetesProvider `talos` ← provider: talos

```yaml
apiVersion: vitistack.io/v1alpha1
kind: KubernetesProvider
metadata:
```

KubernetesCluster – `spec.topology`

🧠 Control plane and 💪 workers

```yaml
topology:
  version: 1.36.5
  controlplane:
    architecture: amd64
    machineClass: small
    provider: kubevirt
    replicas: 1
    storage:
    - class: standard
      path: /var/lib/vitistack/kubevirt
      size: 20Gi
    version: 1.36.5
```

```yaml
  workers:
    nodePools:
    - name: workerpool
      architecture: amd64
      autoscaling:
        enabled: false
        minReplicas: 1
        maxReplicas: 5
        scalingRules: [cpu]
      machineClass: medium
      provider: kubevirt
      replicas: 1
      taint: []
      storage:
      - class: standard
        path: /var/lib/vitistack/kubevirt
        size: 20Gi
      version: 1.36.5
```

KubernetesCluster — `spec.topology`

🧠 Control plane and 💪 workers

```yaml
topology:
  version: 1.36.5
  controlplane:
    architecture: amd64
    machineClass: small
    provider: kubevirt
    replicas: 1
    storage:
    - class: standard
      path: /var/lib/vitistack/kubevirt
      size: 20Gi
    version: 1.36.5
```

```yaml
workers:
  nodePools:
  - name: workerpool
    architecture: amd64
    autoscaling:
      enabled: false
      minReplicas: 1
      maxReplicas: 5
      scalingRules: [cpu]
    machineClass: medium
    provider: kubevirt
    replicas: 1
    taint: []
    storage:
    - class: standard
      path: /var/lib/vitistack/kubevirt
      size: 20Gi
    version: 1.36.5
```

🧩 MachineClass `small` ← machineClass: small

```yaml
apiVersion: vitistack.io/v1alpha1
kind: MachineClass
```

KubernetesCluster – `spec.topology`

🧠 Control plane and 💪 workers

```yaml
topology:
  version: 1.36.5
  controlplane:
    architecture: amd64
    machineClass: small
    provider: kubevirt
    replicas: 1
    storage:
    - class: standard
      path: /var/lib/vitistack/kubevirt
      size: 20Gi
    version: 1.36.5
```

```yaml
workers:
  nodePools:
  - name: workerpool
    architecture: amd64
    autoscaling:
      enabled: false
      minReplicas: 1
      maxReplicas: 5
      scalingRules: [cpu]
    machineClass: medium
    provider: kubevirt
    replicas: 1
    taint: []
    storage:
    - class: standard
      path: /var/lib/vitistack/kubevirt
      size: 20Gi
    version: 1.36.5
```

🖥️ MachineProvider `kubevirt` ← provider

```yaml
apiVersion: vitistack.io/v1alpha1
kind: MachineProvider
```

KubernetesCluster – `spec.topology`

🧠 Control plane and 💪 workers

```yaml
topology:
  version: 1.36.5
  controlplane:
    architecture: amd64
    machineClass: small
    provider: kubevirt
    replicas: 1
    storage:
    - class: standard
      path: /var/lib/vitistack/kubevirt
      size: 20Gi
    version: 1.36.5
```

```yaml
workers:
  nodePools:
  - name: workerpool
    architecture: amd64
    autoscaling:
      enabled: false
      minReplicas: 1
      maxReplicas: 5
      scalingRules: [cpu]
    machineClass: medium
    provider: kubevirt
    replicas: 1
    taint: []
    storage:
    - class: standard
      path: /var/lib/vitistack/kubevirt
      size: 20Gi
    version: 1.36.5
```

KubernetesCluster – `spec.topology`

🧠 Control plane and 💪 workers

```yaml
topology:
  version: 1.36.5
  controlplane:
    architecture: amd64
    machineClass: small
    provider: kubevirt
    replicas: 1
    storage:
    - class: standard
      path: /var/lib/vitistack/kubevirt
      size: 20Gi
    version: 1.36.5
```

```yaml
  workers:
    nodePools:
    - name: workerpool
      architecture: amd64
      autoscaling:
        enabled: false
        minReplicas: 1
        maxReplicas: 5
        scalingRules: [cpu]
      machineClass: medium
      provider: kubevirt
      replicas: 1
      taint: []
      storage:
      - class: standard
        path: /var/lib/vitistack/kubevirt
        size: 20Gi
      version: 1.36.5
```

KubernetesCluster – `spec.topology`

🧠 Control plane and 💪 workers

```yaml
topology:
  version: 1.36.5
  controlplane:
    architecture: amd64
    machineClass: small
    provider: kubevirt
    replicas: 1
    storage:
    - class: standard
      path: /var/lib/vitistack/kubevirt
      size: 20Gi
    version: 1.36.5
```

```yaml
  workers:
    nodePools:
    - name: workerpool
      architecture: amd64
      autoscaling:
        enabled: false
        minReplicas: 1
        maxReplicas: 5
        scalingRules: [cpu]
      machineClass: medium
      provider: kubevirt
      replicas: 1
      taint: []
      storage:
      - class: standard
        path: /var/lib/vitistack/kubevirt
        size: 20Gi
      version: 1.36.5
```

KubernetesCluster — `spec.topology`

🧠 Control plane and 💪 workers

```yaml
topology:
  version: 1.36.5
  controlplane:
    architecture: amd64
    machineClass: small
    provider: kubevirt
    replicas: 1
    storage:
    - class: standard
      path: /var/lib/vitistack/kubevirt
      size: 20Gi
    version: 1.36.5
```

```yaml
  workers:
    nodePools:
    - name: workerpool
      architecture: amd64
      autoscaling:
        enabled: false
        minReplicas: 1
        maxReplicas: 5
        scalingRules: [cpu]
      machineClass: medium
      provider: kubevirt
      replicas: 1
      taint: []
      storage:
      - class: standard
        path: /var/lib/vitistack/kubevirt
        size: 20Gi
      version: 1.36.5
```

🧩 MachineClass `medium` machineClass: medium →

```yaml
apiVersion: vitistack.io/v1alpha1
kind: MachineClass
```

KubernetesCluster – `spec.topology`

🧠 Control plane and 💪 workers

```yaml
topology:
  version: 1.36.5
  controlplane:
    architecture: amd64
    machineClass: small
    provider: kubevirt
    replicas: 1
    storage:
    - class: standard
      path: /var/lib/vitistack/kubevirt
      size: 20Gi
    version: 1.36.5
```

```yaml
  workers:
    nodePools:
    - name: workerpool
      architecture: amd64
      autoscaling:
        enabled: false
        minReplicas: 1
        maxReplicas: 5
        scalingRules: [cpu]
      machineClass: medium
      provider: kubevirt
      replicas: 1
      taint: []
      storage:
      - class: standard
        path: /var/lib/vitistack/kubevirt
        size: 20Gi
      version: 1.36.5
```

🖥️ MachineProvider `kubevirt` provider →

```yaml
apiVersion: vitistack.io/v1alpha1
kind: MachineProvider
```

KubernetesCluster – `spec.topology`

🧠 Control plane and 💪 workers

```yaml
topology:
  version: 1.36.5
  controlplane:
    architecture: amd64
    machineClass: small
    provider: kubevirt
    replicas: 1
    storage:
    - class: standard
      path: /var/lib/vitistack/kubevirt
      size: 20Gi
    version: 1.36.5
```

```yaml
  workers:
    nodePools:
    - name: workerpool
      architecture: amd64
      autoscaling:
        enabled: false
        minReplicas: 1
        maxReplicas: 5
        scalingRules: [cpu]
      machineClass: medium
      provider: kubevirt
      replicas: 1
      taint: []
      storage:
      - class: standard
        path: /var/lib/vitistack/kubevirt
        size: 20Gi
      version: 1.36.5
```

KubernetesCluster – `spec.topology`

🧠 Control plane and 💪 workers

```yaml
topology:
  version: 1.36.5
  controlplane:
    architecture: amd64
    machineClass: small
    provider: kubevirt
    replicas: 1
    storage:
    - class: standard
      path: /var/lib/vitistack/kubevirt
      size: 20Gi
    version: 1.36.5
```

```yaml
  workers:
    nodePools:
    - name: workerpool
      architecture: amd64
      autoscaling:
        enabled: false
        minReplicas: 1
        maxReplicas: 5
        scalingRules: [cpu]
      machineClass: medium
      provider: kubevirt
      replicas: 1
      taint: []
      storage:
      - class: standard
        path: /var/lib/vitistack/kubevirt
        size: 20Gi
      version: 1.36.5
```

🚀 kubectl apply



```
# 1. k8s namespace
kubectl create namespace team-cool

# 2. Network 🌐
kubectl apply -f manifests/networknamespace.yaml -n team-cool

⧗ wait for the network to be ready

# 3. Then the cluster
kubectl apply -f manifests/kubernetescluster.yaml -n team-cool
```

🚀 kubectl apply



```
# 1. k8s namespace
kubectl create namespace team-cool

# 2. Network 🌐
kubectl apply -f manifests/networknamespace.yaml -n team-cool

⧗ wait for the network to be ready

# 3. Then the cluster
kubectl apply -f manifests/kubernetescluster.yaml -n team-cool
```

```
# Watch it come alive 👀
watch -n 1 kubectl get networknamespace,kubernetescluster,machines -n team-cool
```

```
# Get kubeconfig for new cluster
viti kc login -o=.
```

```
# Set KUBECONFIG
export KUBECONFIG=./kubeconfig-supercluster-1337
```

```
# Get nodes
kubectl get nodes
```

🚀 kubectl apply



```
# 1. k8s namespace
kubectl create namespace team-cool

# 2. Network 🌐
kubectl apply -f manifests/networknamespace.yaml -n team-cool

⧗ wait for the network to be ready

# 3. Then the cluster
kubectl apply -f manifests/kubernetescluster.yaml -n team-cool
```

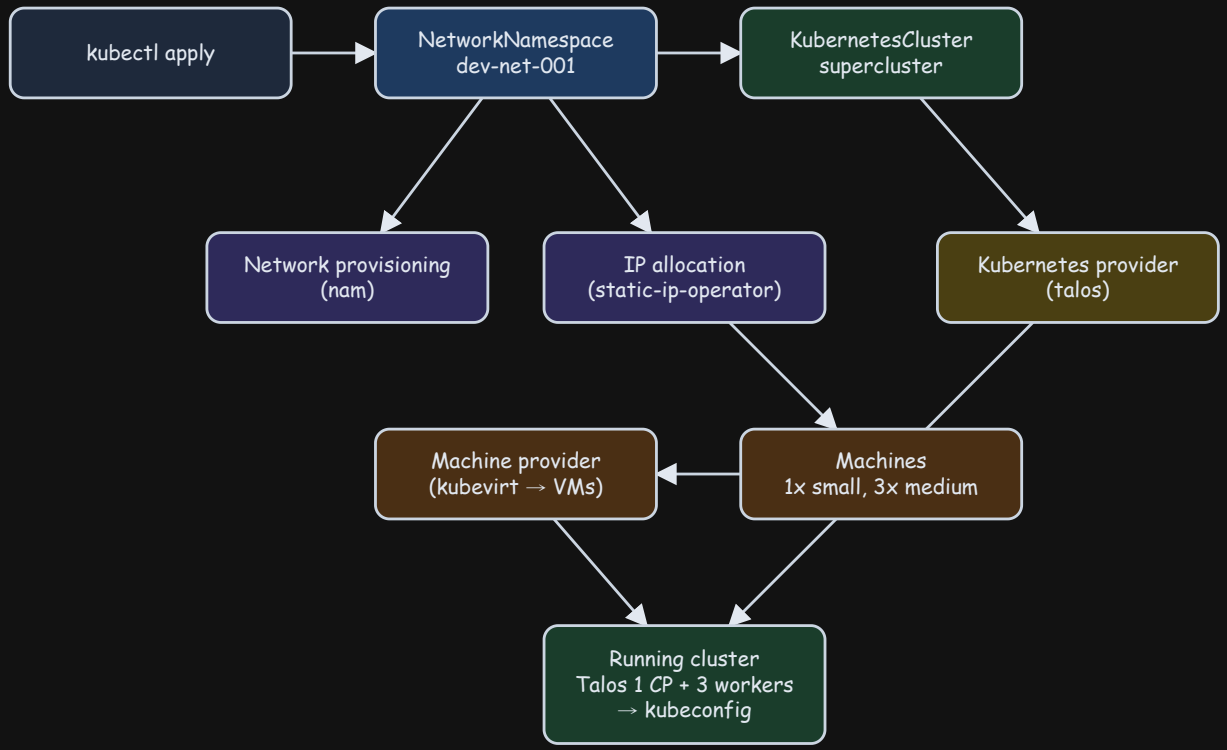
```
# Watch it come alive 👀
watch -n 1 kubectl get networknamespace,kubernetescluster,machines -n team-cool
```

```
# Get kubeconfig for new cluster
viti kc login -o=.
```

```
# Set KUBECONFIG
export KUBECONFIG=./kubeconfig-supercluster-1337
```

```
# Get nodes
kubectl get nodes
```

How it fits together

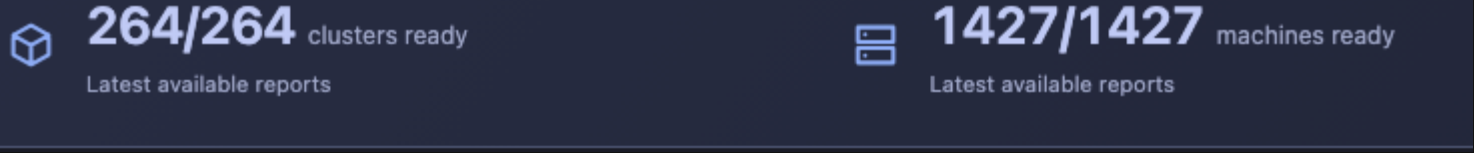




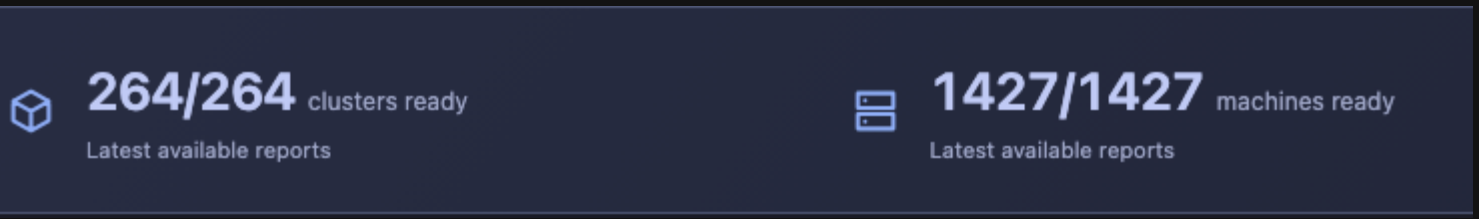
Tested at scale



Tested at scale



Tested at scale



Regions: 3 (west, central, east, soon north)

Tested at scale

264/264 clusters ready

Latest available reports

1427/1427 machines ready

Latest available reports

Regions: 3 (west, central, east, soon north)

But also home lab:



💬 Experiences using Vitistack



💬 Experiences using Vitistack



- 👍 Clusters as YAML → GitOps, review, repeatability

💬 Experiences using Vitistack



- 👍 Clusters as YAML → GitOps, review, repeatability
- 👍 Pluggable providers – swap KubeVirt / Proxmox / physical

💬 Experiences using Vitistack



- 👍 Clusters as YAML → GitOps, review, repeatability
- 👍 Pluggable providers – swap KubeVirt / Proxmox / physical
- 👍 Talos: immutable, API-driven nodes

💬 Experiences using Vitistack



- 👍 Clusters as YAML → GitOps, review, repeatability
- 👍 Pluggable providers – swap KubeVirt / Proxmox / physical
- 👍 Talos: immutable, API-driven nodes
- 👍 Scales fast

💬 Experiences using Vitistack



- 👍 Clusters as YAML → GitOps, review, repeatability
- 👍 Pluggable providers – swap KubeVirt / Proxmox / physical
- 👍 Talos: immutable, API-driven nodes
- 👍 Scales fast
- 🤔 Lessons learned …
  - Naming standards
  - Kea DHCP was not good
  - Need of overview beyond regions

💬 Experiences using Vitistack



- 👍 Clusters as YAML → GitOps, review, repeatability
- 👍 Pluggable providers – swap KubeVirt / Proxmox / physical
- 👍 Talos: immutable, API-driven nodes
- 👍 Scales fast
- 🤔 Lessons learned …
  - Naming standards
  - Kea DHCP was not good
  - Need of overview beyond regions
- 🤔 What was hard …
  - Kubernetes knowledge mountain
  - Golang programming
  - Integrations with custom parts

🔮 The future of Vitistack



🔮 The future of Vitistack



- 🧩 Template based machine config

🔮 The future of Vitistack



- 🧩 Template based machine config
- 🔌 More providers (machines and Kubernetes distributions, europe based)

🔮 The future of Vitistack



- 🧩 Template based machine config
- 🔌 More providers (machines and Kubernetes distributions, europe based)
- 📀 More OSes (debian, ubuntu, etc)

🔮 The future of Vitistack



- 🧩 Template based machine config
- 🔌 More providers (machines and Kubernetes distributions, europe based)
- 📀 More OSes (debian, ubuntu, etc)
- 📈 Autoscaling of node pools

🔮 The future of Vitistack



- 🧩 Template based machine config
- 🔌 More providers (machines and Kubernetes distributions, europe based)
- 📀 More OSes (debian, ubuntu, etc)
- 📈 Autoscaling of node pools
- 💾 Automatic backup & restore (EtcdBackup)

🔮 The future of Vitistack



- 🧩 Template based machine config
- 🔌 More providers (machines and Kubernetes distributions, europe based)
- 📀 More OSes (debian, ubuntu, etc)
- 📈 Autoscaling of node pools
- 💾 Automatic backup & restore (EtcdBackup)
- 📚 Better docs and architecture guides

🔮 The future of Vitistack



- 🧩 Template based machine config
- 🔌 More providers (machines and Kubernetes distributions, europe based)
- 📀 More OSes (debian, ubuntu, etc)
- 📈 Autoscaling of node pools
- 💾 Automatic backup & restore (EtcdBackup)
- 📚 Better docs and architecture guides
- 🤝 Community & contributions



Thanks! 🙏

❓ Questions & 💡 feedback welcome

🌐 vitistack.io

📖 docs.vitistack.io / docs.vitistack.io/getting_started

🐙 github.com/vitistack

👋 Roger Westerbø · ✉️ roger.westerbo@gmail.com